

SafeLine WAF Home Lab

Protecting DVWA with HTTPS, SQL Injection Blocking, Rate Limiting,
Authentication, and Custom Deny Rules

Cybersecurity Home Lab Project Report

Navdeep Singh

Completed: October 2026

Scope: isolated home lab systems owned and controlled by the author. All attack simulation was performed only against the intentionally vulnerable DVWA environment.

1. Executive Summary

I built this project to understand how a Web Application Firewall (WAF) works in front of a vulnerable web application. The lab used Kali Linux as the testing machine, Ubuntu Server as the host, DVWA as the intentionally vulnerable application, and SafeLine WAF as the HTTPS reverse proxy and security layer.

The main goal was not just to install the tools. I wanted to verify each control with evidence. I tested SQL injection blocking, HTTP rate limiting, WAF-level authentication, and a source-IP deny rule. For each test, I checked what the client saw and then confirmed the result in SafeLine where possible.

The project followed the structure of the user-supplied "Web Application Firewall Home Lab using SafeLine WAF" guide by Royden Rebello (The Social Dork), while this report documents my own implementation, troubleshooting, and results.

Project Outcomes

- SafeLine successfully served DVWA through HTTPS and reverse-proxied requests to Apache on port 8080.
- A basic SQL injection attempt was blocked and logged as SQL Inj by SafeLine.
- A configured 100 requests / 10 seconds threshold was triggered during a controlled ApacheBench test.
- SafeLine authentication successfully added a sign-in step before traffic reached DVWA.
- A custom deny rule blocked the Kali source IP and returned an Access Forbidden page.
- Temporary blocking controls were disabled after testing so the lab remained usable.

2. Objectives and Scope

The objectives for this lab were:

- Build a small multi-VM web-security environment using VirtualBox.
- Run a deliberately vulnerable web application on Ubuntu Server.
- Place SafeLine WAF in front of DVWA as a reverse proxy.
- Use HTTPS with a self-signed certificate.
- Generate controlled security-test traffic from Kali Linux.
- Confirm that WAF protections work by reviewing browser behavior and SafeLine logs.
- Keep the lab isolated from production systems and avoid publishing credentials or private-key material.

3. Lab Architecture

Both virtual machines used bridged networking so Kali and Ubuntu were on the same local lab network. Kali sent HTTPS requests to dvwa.local. SafeLine listened on port 443 and forwarded allowed requests to Apache/DVWA on port 8080.

Component	Role	Lab address	Notes
Kali Linux	Testing client	192.168.2.102	Browser, ping, ApacheBench

Ubuntu Server	Application/WAF host	192.168.2.101	SafeLine, Apache, PHP, MySQL, DVWA
SafeLine WAF	Reverse proxy / protection	443; management 9443	HTTPS inspection and security controls
Apache + DVWA	Backend application	192.168.2.101:8080/DVWA/	Intentionally vulnerable training app
dvwa.local	Local hostname	Mapped to 192.168.2.101	Static /etc/hosts entry on Kali

The addresses are RFC1918 private lab addresses and are not publicly routable.

4. Build and WAF Configuration

4.1 Ubuntu, DVWA, and Apache

Ubuntu Server hosted Apache, PHP, MySQL, DVWA, Docker, and SafeLine. DVWA was installed under /var/www/html/DVWA. Apache was moved to port 8080 so SafeLine could accept client HTTPS traffic on port 443 and forward it to the backend.

A direct backend check confirmed that the DVWA path responded and redirected unauthenticated users to its login page.

```
curl -I http://192.168.2.101:8080/DVWA/
```

4.2 Local Name Resolution

Kali did not have a DNS record for dvwa.local, so I added a static hosts-file entry. This gave the lab a simple local hostname without depending on an external DNS service.

```
192.168.2.101 dvwa.local
```

4.3 HTTPS Certificate and SafeLine Reverse Proxy

I created a self-signed certificate for the lab and imported it into SafeLine. The SafeLine application used dvwa.local on HTTPS/443 and forwarded allowed requests to http://192.168.2.101:8080.

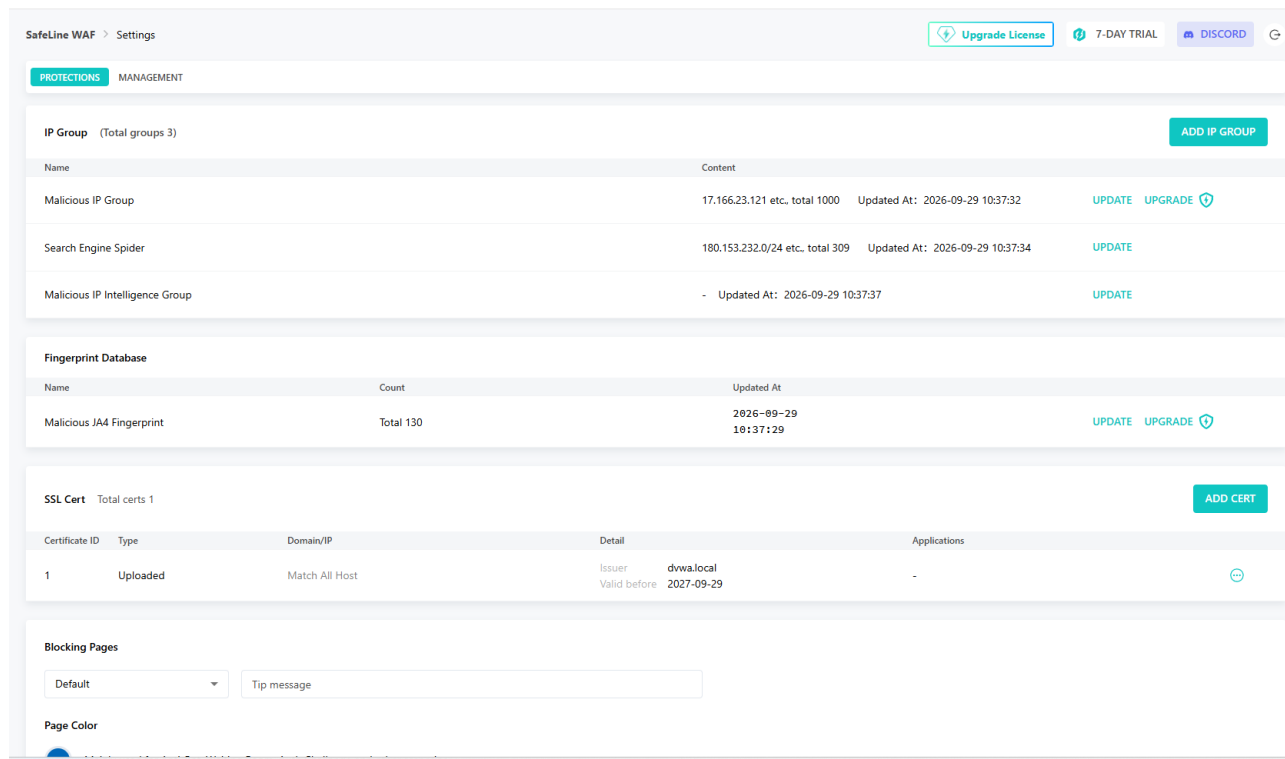


Figure 1. Self-signed certificate successfully imported into SafeLine.

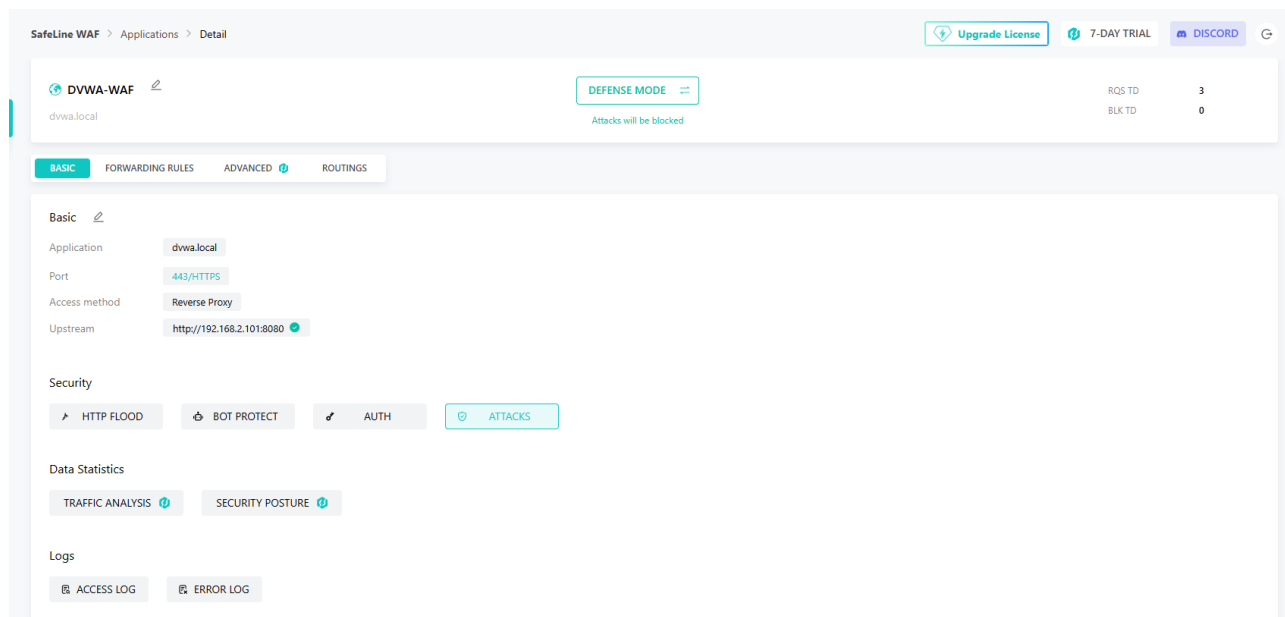


Figure 2. SafeLine application configured for HTTPS reverse proxy to Apache on port 8080.

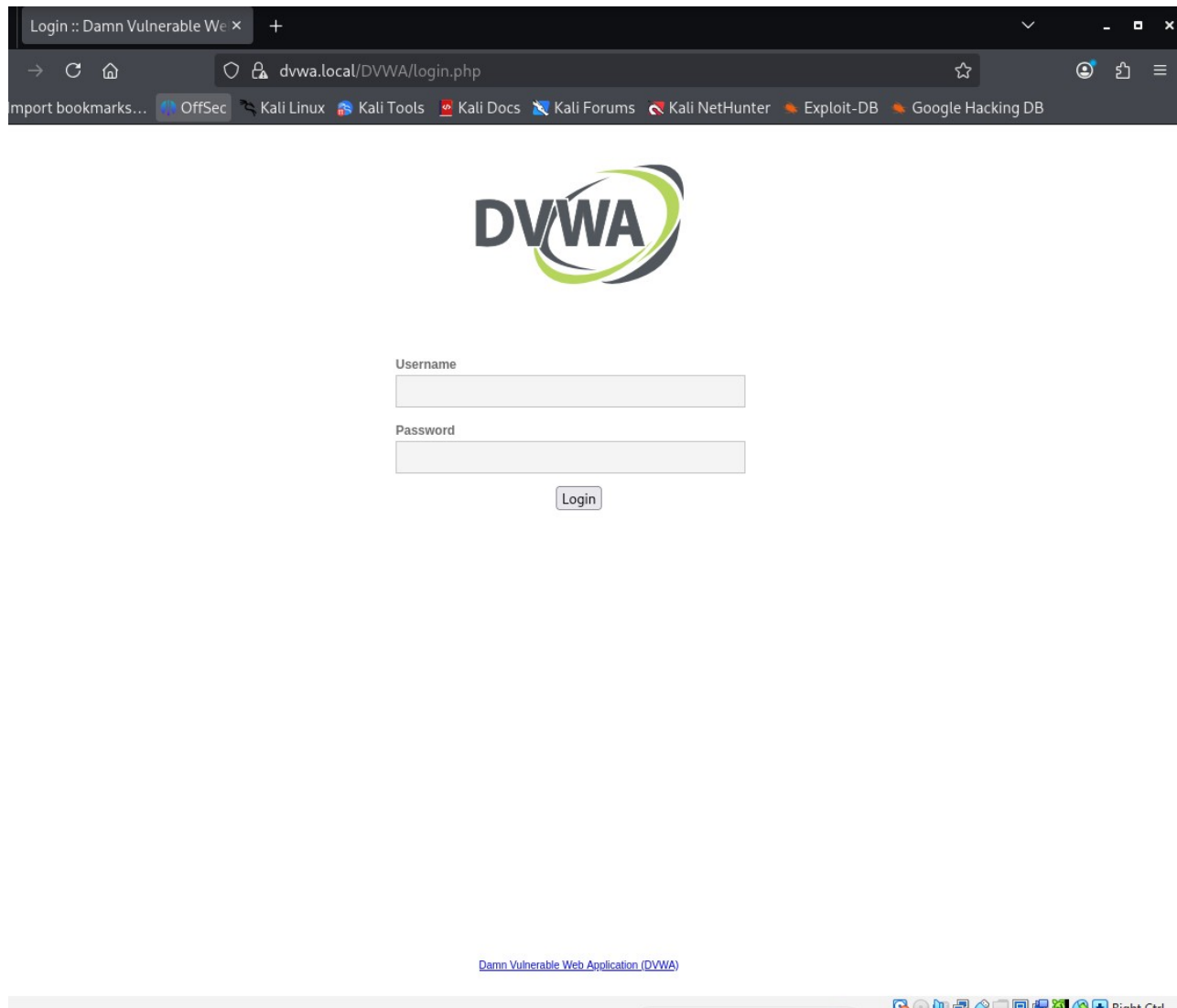


Figure 3. DVWA login page reached through SafeLine using the dvwa.local HTTPS path.

5. Security Validation Tests

The following tests were performed only against this isolated home lab. Each result was treated as successful only after the observed behavior matched the configured security control.

5.1 SQL Injection Protection

DVWA Security was set to Low so the backend was intentionally vulnerable. I then submitted a basic SQL injection test string from Kali. SafeLine returned an Access Forbidden page, and the attack log recorded the request as SQL Inj with the action Blocked.

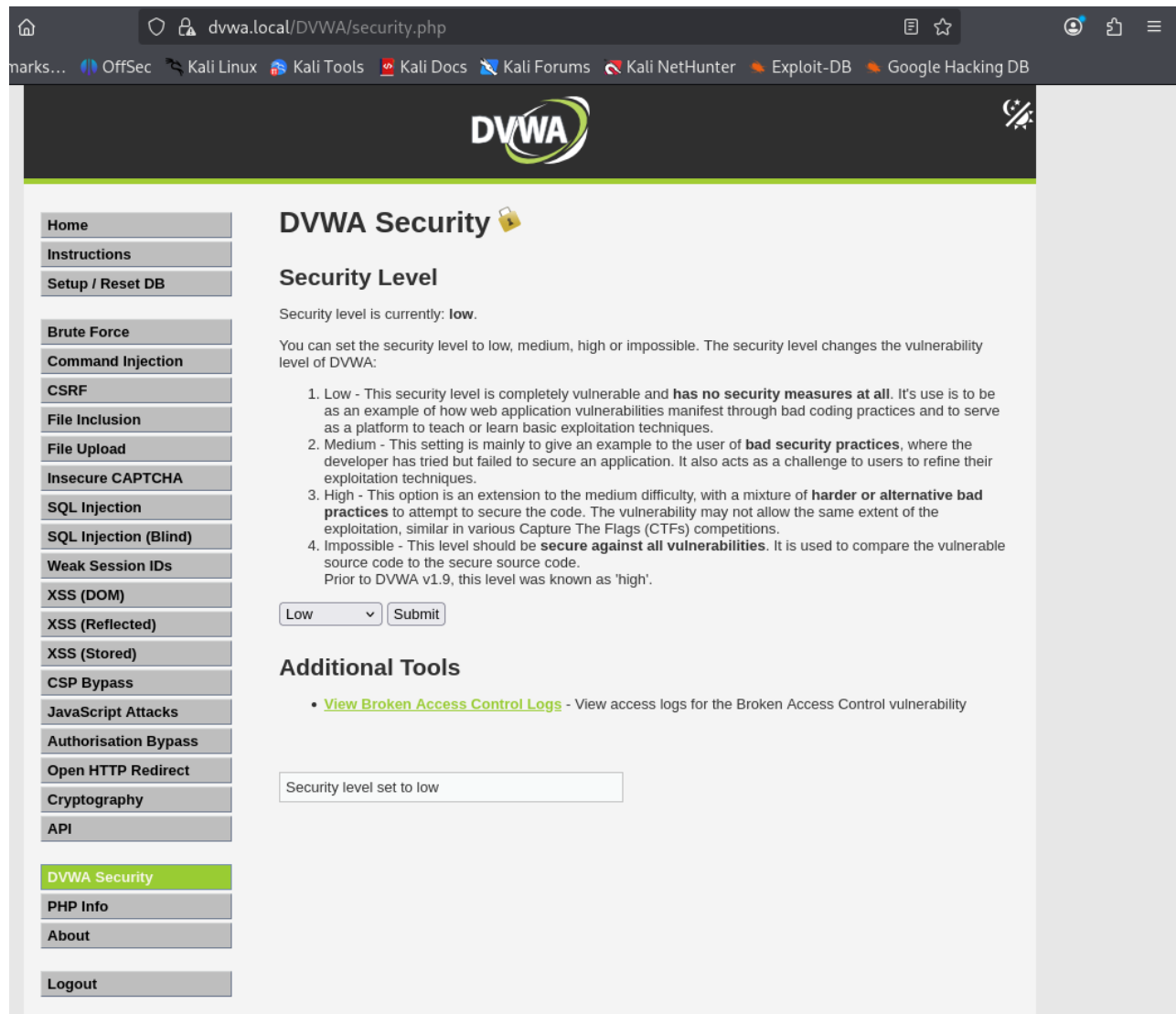


Figure 4. DVWA Security set to Low before the WAF test.

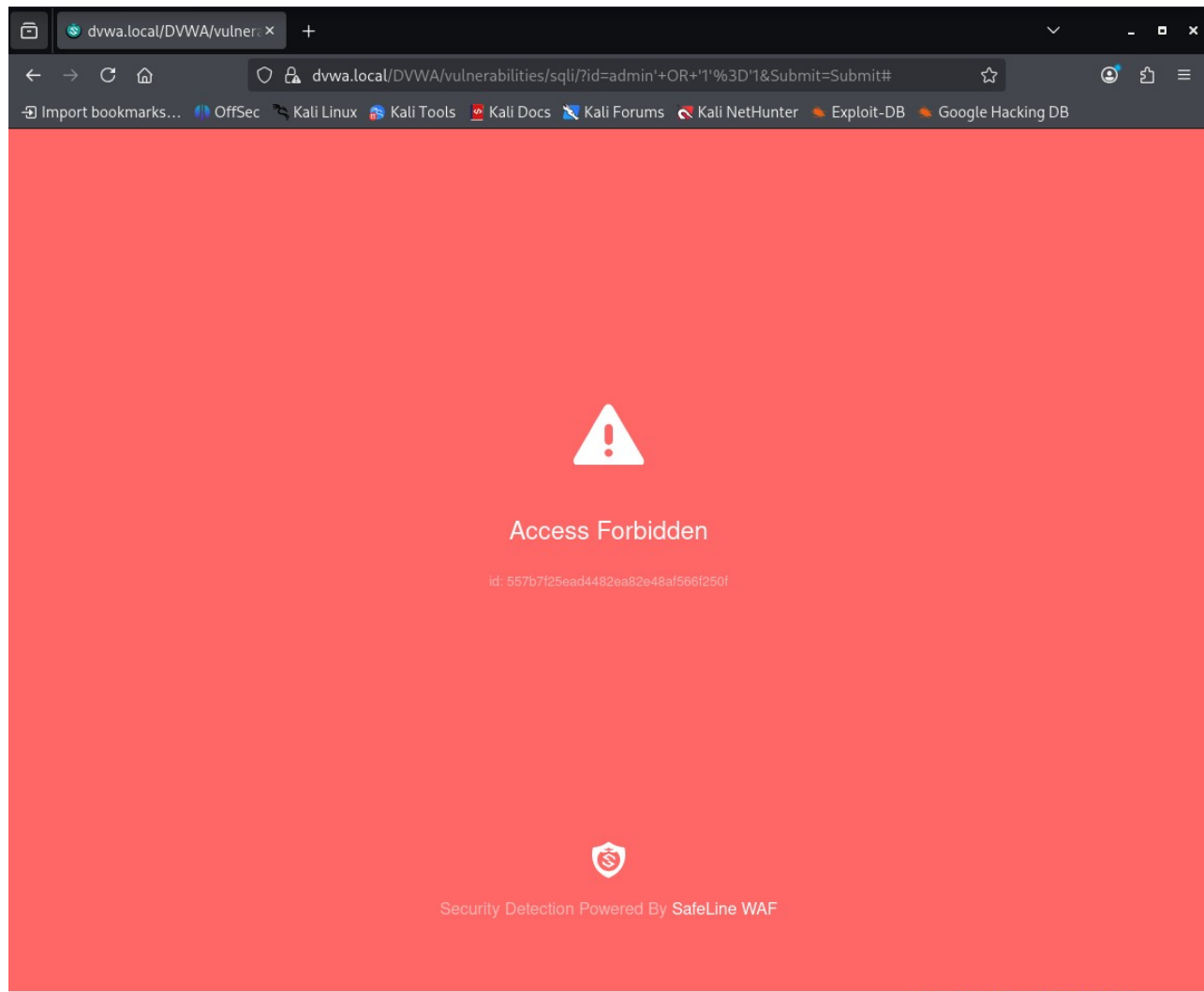


Figure 5. SafeLine Access Forbidden response to the SQL injection attempt.

The screenshot shows the SafeLine WAF interface. At the top, there's a navigation bar with 'SafeLine WAF > Attacks'. Below this, a 'Semantic Analysis' section explains its function and lists three modes: Audit, Balance, and Strict. The main area displays a log entry for a blocked request. The log entry shows the following details:

Action	IP Addr	Domain	Port	Time	Attack Type
Blocked	192.168.2.102	dvwa.local	443	2026-09-30 11:41:47	SQL Inj

The log entry also includes the URL: `https://dvwa.local/DVWA/vulnerabilities/sql/?id=admin%27+OR+%271%27%3D%271&Submit=Submit`. The interface includes filters for Action, IP Addr, Domain, Port, and Time, along with buttons for 'EXPORT', 'REFRESH', and 'MORE'. The bottom of the log entry shows pagination: '20 per page, total 1'.

Figure 6. SafeLine log showing the SQL injection event and Blocked action.

Result: The test confirmed that SafeLine detected and blocked the SQL injection request before it was processed by the vulnerable DVWA backend.

5.2 HTTP Flood / Rate Limiting

I enabled Basic Access Limit with a threshold of 100 requests within 10 seconds. The configured response was an Anti-Bot challenge for 60 minutes. To test the control, I used ApacheBench from Kali with 150 total requests and 10 concurrent requests against my own DVWA endpoint.

`ab -n 150 -c 10 https://dvwa.local/DVWA/`

The screenshot shows the 'Settings' page for 'HTTP Flood > Rate Limiting'. It features three sections: 'Access Limiting', 'Attack Limiting', and 'Error Limiting'. The 'Access Limiting' section is currently active and shows the 'Basic Access Limit' toggle is turned on. The description states: 'An IP that makes 100 requests within 10 seconds will require Anti-Bot challenge when accessing again within the next 60 minutes.' The 'Attack Limiting' section shows the 'Basic Attack Limit' toggle is turned off. The 'Error Limiting' section shows the 'Basic Error Limit' toggle is turned off. Each section has an 'EDIT' button.

Figure 7. Basic Access Limit enabled with the configured threshold.

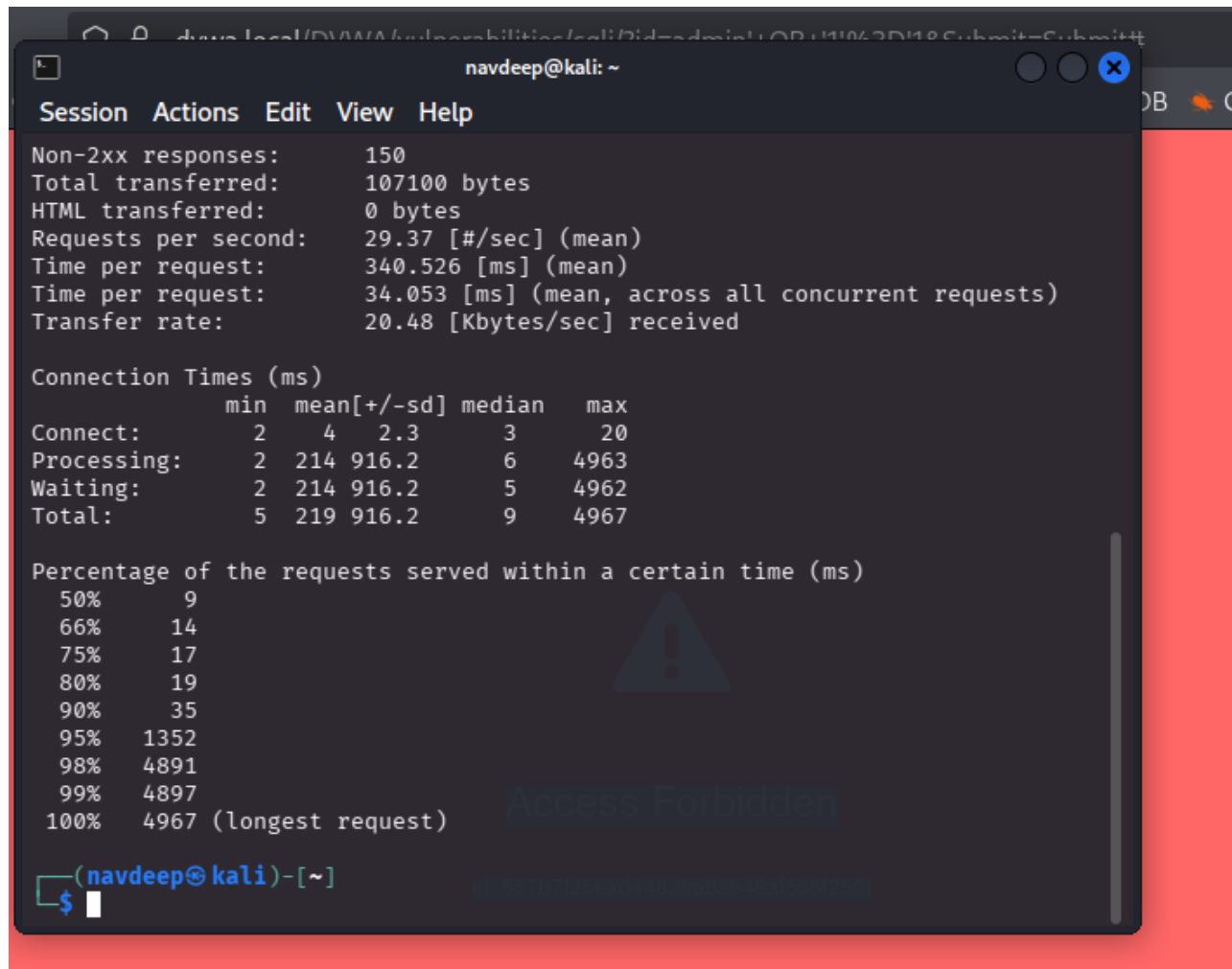


Figure 8. Controlled ApacheBench traffic generated from Kali.

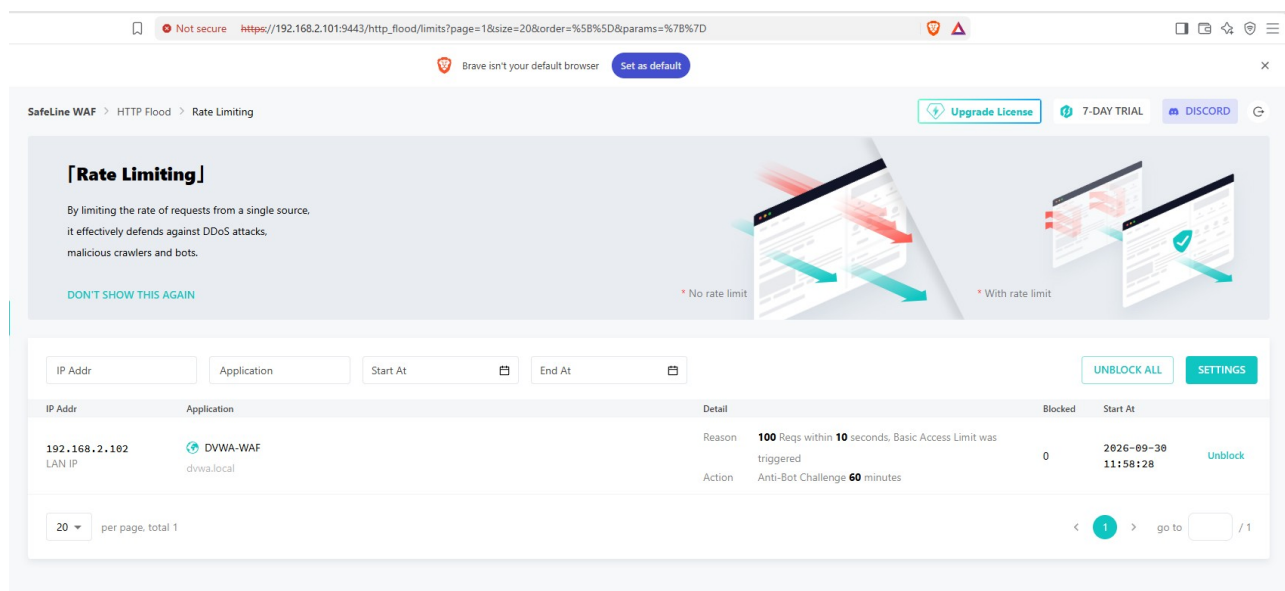


Figure 9. SafeLine event showing the 100 requests / 10 seconds threshold was triggered.

Result: The rate-limiting control triggered as configured. The action was an Anti-Bot challenge, so I did not describe this result as a direct IP block.

5.3 WAF Authentication

I created a local SafeLine test user and enabled Simple Auth with account-password authentication for the DVWA application. When Kali accessed dvwa.local, SafeLine displayed its own sign-in page before allowing access to the DVWA application.

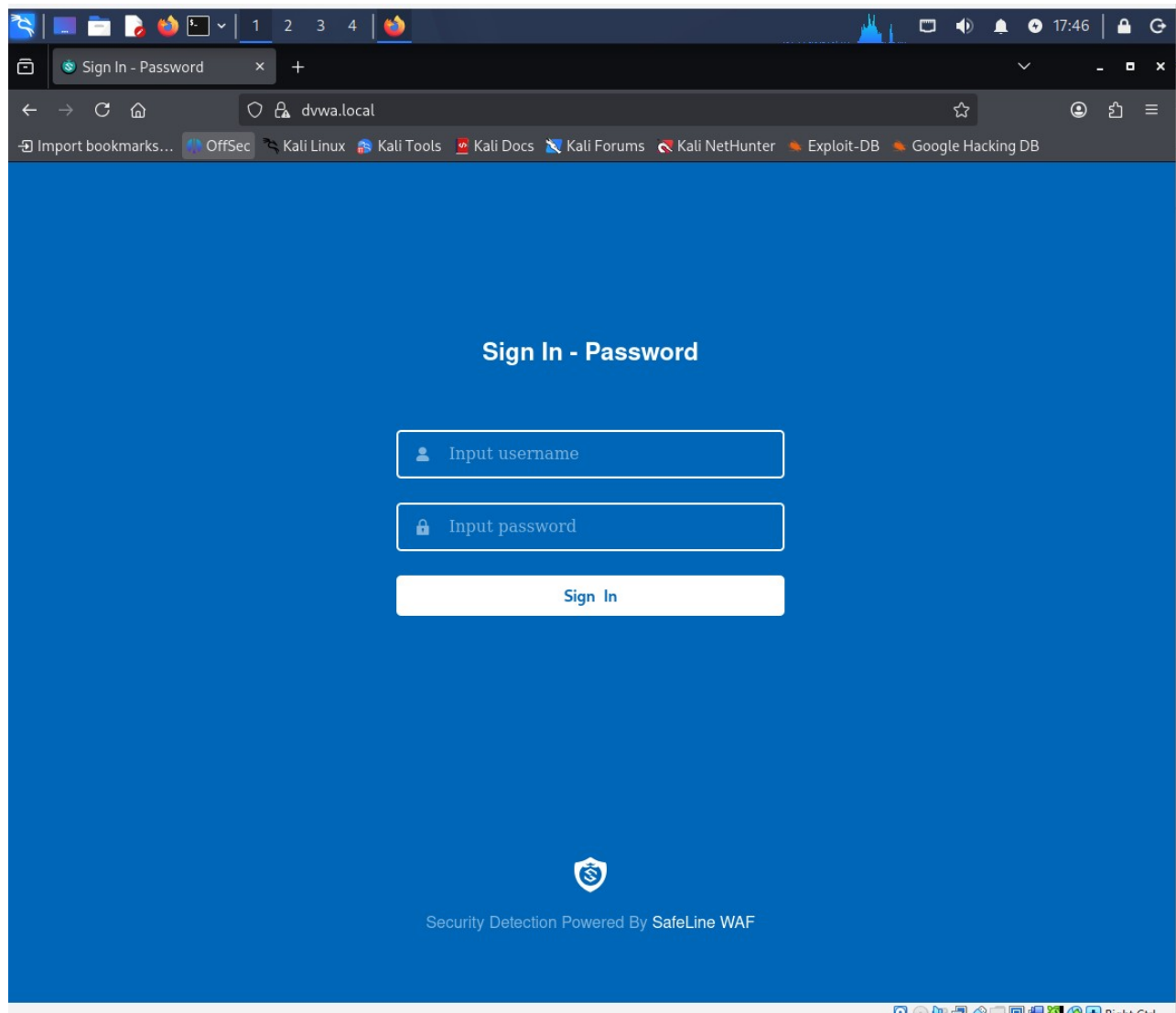


Figure 10. SafeLine authentication challenge presented before access to DVWA.

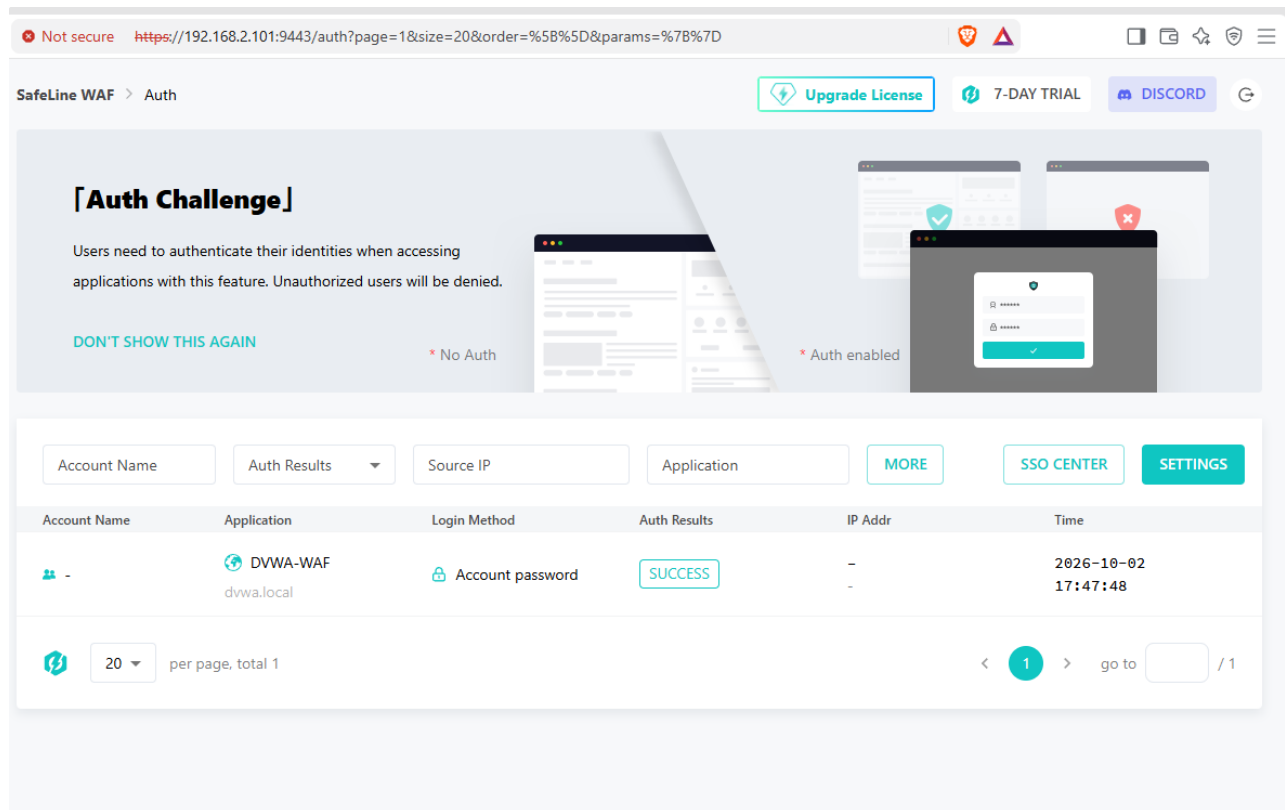


Figure 11. SafeLine log recording an Account password authentication result of SUCCESS.

Result: SafeLine successfully added an authentication layer in front of DVWA. Credentials are intentionally omitted from this report.

5.4 Custom Deny Rule

For the final access-control test, I created a blacklist rule matching the Kali VM source IP (192.168.2.102). The rule was enabled and inserted first so it would be evaluated with high priority.

Add rules

Allow

Deny Rule

Name *

Block-kali-IP

Insert Position

First

Match Target

Source IP

Operator *

Equals

Content *

192.168.2.102 ×

e.g: 192.168.10.10

ADD AN AND CONDITION

REMOVE THIS CONDITION GROUP

ADD AN OR CONDITION

☒ Enabled

CANCEL

SUBMIT

Figure 12. Custom deny rule configured for the Kali source IP.

Navdeep Singh | SafeLine WAF Home Lab | Cybersecurity Portfolio

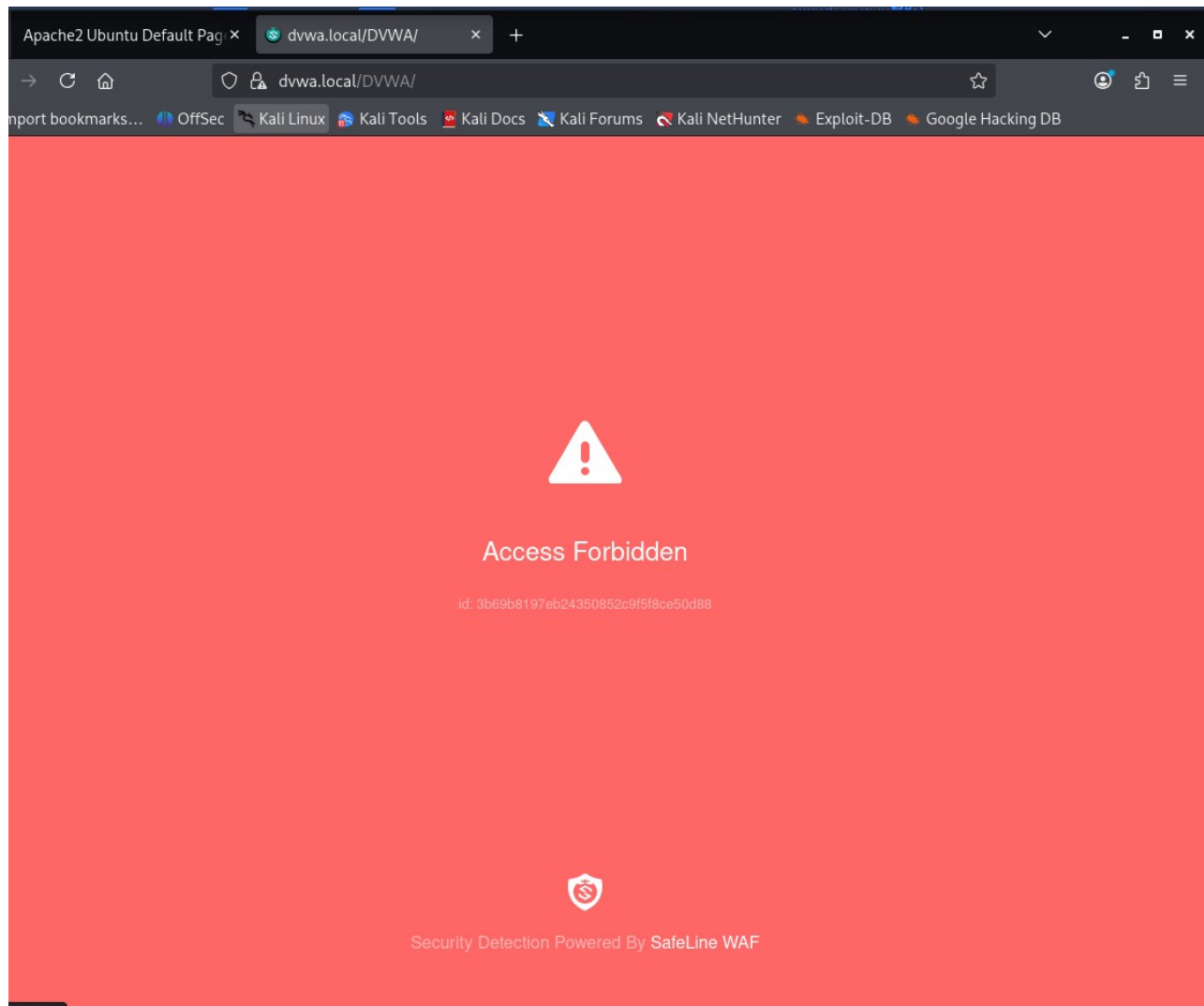


Figure 13. Kali receives Access Forbidden after the deny rule is enabled.

After verifying the block, I disabled the temporary rule so I would not remain locked out of the lab during future sessions.

6. Troubleshooting and Lessons Learned

Issue	What I did / learned
Kali networking	Kali initially used NAT while Ubuntu was on the 192.168.2.x network. I switched Kali to a Bridged Adapter, received 192.168.2.102, and verified reachability to Ubuntu.
Local hostname resolution	dvwa.local did not resolve automatically. I added a static /etc/hosts mapping on Kali. This is local name resolution, not a full DNS server.
Apache default page	Opening https://dvwa.local/ showed the Apache default page because DVWA was installed under /DVWA/. The working protected application path was https://dvwa.local/DVWA/.
DVWA / MySQL compatibility	The DVWA database setup initially failed with the installed MySQL version. I made a small compatibility adjustment to the DVWA MySQL setup code and then completed the database setup.
SafeLine UI appeared stuck	During one application submission, the SafeLine page kept loading. I checked Docker container health before changing anything else and found the SafeLine services were running; the application had actually been created.
VM soft-lockup warnings	Ubuntu showed kernel soft-lockup warnings under heavier SafeLine/Docker load. I used graceful shutdowns and verified container health after reboot before continuing.

7. Security and Publishing Review

Before preparing this project for GitHub, I reviewed the screenshots and removed evidence that exposed unnecessary secrets. I chose to exclude sensitive screenshots instead of trying to publish them with partial redactions when the evidence was not needed.

- SafeLine administrator credentials are not included.
- Certificate private-key contents are not included.
- The SafeLine test-user password is not included.
- Database credentials are not included.
- Screenshots containing PHPSESSID or other active session-cookie values were excluded.
- Temporary certificate-key copies used during setup were deleted from the Windows Downloads folder and from the temporary Ubuntu home-directory location.

- Private 192.168.2.x lab addresses remain visible because they are non-routable and useful for understanding the lab network.

8. Skills Demonstrated

- VirtualBox VM configuration and bridged networking
- Ubuntu Server administration and basic Linux CLI usage
- Apache/PHP/MySQL/DVWA deployment and troubleshooting
- Docker-hosted SafeLine WAF administration
- HTTPS certificate handling in a lab environment
- Reverse proxy concepts
- Web Application Firewall testing
- SQL injection detection validation
- HTTP rate limiting / anti-bot testing
- Authentication gateway configuration
- Source-IP access-control rules
- Security log review and evidence-based verification
- Safe documentation and secret-handling practices

9. Conclusion and Next Steps

This lab gave me practical experience with the traffic flow between a client, a WAF, and a backend web application. The most useful part was comparing what the browser showed with what SafeLine logged. That helped me understand the difference between simply generating traffic and actually proving which security control handled it.

Useful next steps for this lab would be:

- Test additional DVWA vulnerabilities such as XSS and command injection and compare SafeLine behavior.
- Add OWASP Juice Shop as another vulnerable application.
- Send WAF or Apache events to a SIEM for investigation practice.
- Compare different SafeLine detection modes and rule settings.
- Add IDS/IPS or firewall telemetry for a more complete multi-source security investigation lab.

10. Reference

Royden Rebello (The Social Dork), "Web Application Firewall Home Lab using SafeLine WAF." User-supplied lab guide used as the reference roadmap for the project.

The source guide recommends keeping the homelab isolated from production networks, regularly updating Kali/Ubuntu/SafeLine, and reviewing logs on each layer when troubleshooting.